

AI Capacity Forecaster

Proactive Infrastructure Capacity Planning & Automated SRE Operations Cockpit

Author: Moin Hasan Repository: github.com/moin85m/ai-capacity-forecaster Tech: Python 3.12 • Streamlit • Plotly • Gemini API

1. Problem Statement & Executive Summary

Modern cloud-native and Kubernetes clusters frequently face **reactive incident management**. Traditional monitoring tools (Prometheus, Datadog, CloudWatch) alert on-call engineers *after* thresholds (CPU > 90%, Memory > 85%) are already breached, resulting in latency spikes, OOM-killed container restarts, and customer-facing degradation.

AI Capacity Forecaster solves this by shifting operations from reactive alerts to **deterministic trend extrapolation and proactive remediation**, calculating exact time-to-saturation hours ahead of impact.

2. System Architecture & Telemetry Pipeline



3. Architectural Decision Records (ADRs) & Engineering Trade-Offs

ADR-01: Explainable Linear Extrapolation vs. Deep Learning (LSTM / Transformers)

Context: Production reliability tools frequently explore complex deep time-series models (Prophet, LSTMs, Temporal Fusion Transformers).

Decision: Implemented least-squares linear trend regression with configurable lookback windows ($W \in [4, 24]$ hrs) and R^2 goodness-of-fit scoring.

Engineering Rationale:

- **Explainability & Trust:** In high-stress production incidents, on-call SREs require transparent mathematical causality. The calculation $ETA = (90\% - Current) / Slope$ is intuitive, auditable, and easily verifiable in real time.
- **Sub-Millisecond Execution:** Executes in-memory without requiring dedicated GPU infrastructure, model artifact weights, or heavy background training pipelines.
- **Short-Horizon Stability:** For 4–12 hour operational horizons, linear slopes prevent the overfitting and false seasonality traps that deep learning models introduce on noisy telemetry.

ADR-02: Zero-Dependency Dual-Mode Reliability Pattern

Context: Mission-critical infrastructure tools must never introduce external single points of failure (SPOFs).

Decision: Architected the platform around two distinct execution engines:

- **Mode A (Deterministic Engine):** 100% offline rule-based heuristics requiring zero API keys, internet connectivity, or credentials.
- **Mode B (AI-Assisted LLM Mode):** Optional integration with Google Gemini or OpenAI to synthesize incident narratives and long-term architectural advice when credentials are present.

Engineering Rationale: Guarantees that during air-gapped operations, network partitions, or external API rate-limiting, the capacity forecaster continues operating with zero degradation.

ADR-03: Strict Separation of Observed Telemetry vs. AI Recommendations

Decision: Distinct visual badges strictly segregate [Observed Telemetry] (unmodified sensor data), [Deterministic Engine Logic] (mathematical regression slopes), and [AI Operations Advisor] (generative narratives).

Engineering Rationale: Complies with enterprise SRE observability standards and completely prevents alert confusion or hallucinated metric interpretations.

4. Multi-Metric Correlation & Scenario Simulation

The platform models multi-vector infrastructure stresses across Compute (CPU %), Memory (RAM %), Network (Bandwidth %), and Ingress Request Rate vs. Pod Replicas. It includes 4 simulation profiles:

- **Steady Workload Growth:** Simulates multi-day scaling toward capacity saturation.
- **Monotonic Memory Leak:** Constant request volume with continuous uncollected heap consumption leading to predicted OOM-kills.
- **Sudden Traffic Spike:** Exponential traffic influx outpacing autoscaling replica provisioning.
- **Stable Operations:** Confirms healthy baseline operating headroom (>40% buffer).

5. Actionable SRE Runbooks & Remediation Outputs

When risks escalate to `HIGH` or `CRITICAL`, the system automatically generates executable CLI commands:

```
# 1. Trigger Kubernetes Horizontal Scale-Out:
kubectl scale deployment/api-gateway --replicas=6

# 2. Adjust Horizontal Pod Autoscaler targets proactively:
kubectl patch hpa api-gateway -p '{"spec":{"targetCPUUtilizationPercentage":70}}'

# 3. Check for Out-Of-Memory (OOM-Killed) pod terminations:
kubectl get events --field-selector reason=OOMKilled -A

# 4. Enforce Ingress Token-Bucket Rate Limiting to protect network interfaces:
kubectl annotate ingress api-ingress nginx.ingress.kubernetes.io/limit-rps='150' --overwrite
```

6. Technology Stack & Implementation Summary

Category	Technology	Implementation Role
Languages & Runtimes	Python 3.12 / 3.11	Vectorized analytics, least-squares algebra, and backend runtime
Dashboard Framework	Streamlit	Reactive SRE operations center with custom dark UI styling
Data Visualization	Plotly Graph Objects	Interactive multi-axis time series, projection bands, and threshold lines
AI Integration	Google Gemini 1.5 Flash / OpenAI GPT-4o-mini	Incident narrative synthesis and architectural recommendations
Operations & Cloud	Kubernetes HPA, AWS ASG	Remediation runbooks and capacity headroom management

7. Quality Assurance & Automated Testing

Automated test suite (tests/test_pipeline.py) validates schema compliance, least-squares slope accuracy, and risk boundary conditions with 100% pass rate:

```
$ python3 tests/test_pipeline.py
Testing data loading... Loaded 48 telemetry rows.
Testing forecasting engine...
  [cpu_percent] Current: 92.1%, Slope: +0.78%/h, Critical ETA: Threshold already exceeded
  [memory_percent] Current: 87.4%, Slope: +0.65%/h, Critical ETA: ~3h 58m
  [network_percent] Current: 83.0%, Slope: +0.83%/h, Critical ETA: ~8h 28m
Testing risk classification... Overall System Risk: CRITICAL (Dominant: CPU Utilization)
ALL SMOKE TESTS PASSED SUCCESSFULLY! 
```